

Sequence data types

Reading

- [PY4E chapter 6](#)
- [PY4E chapter 8](#)
- [PY4E chapter 10](#)

Overview

The common theme of this lecture is that we look at built-in Python data types that represent *ordered sequences* of items. There are three main types: `str`, `list` and `tuple`. In many respects they behave similarly in Python: you can iterate over their elements in the correct order using `for` loops, and you can perform the same types of **indexing** and **slicing** (which will be discussed soon). To summarize, the data types differ in the following way:

Type	Mutable?	Contain
<code>str</code>	no	letters
<code>list</code>	yes	anything
<code>tuple</code>	no	anything

A **mutable** data type can have its value changed after it has been created. We will return to this topic later, because this property has far-reaching implications on programming style.

We begin our discussion with strings, the `str` data type, because it is the most specialized of these data types and of course heavily used when working with most types language data.

Strings

We have seen strings before!

- Single-quoted strings

```
In [1]: 'Python is great!'
```

```
Out[1]: 'Python is great!'
```

- Double-quoted strings

```
In [2]: "Python is great!"
```

```
Out[2]: 'Python is great!'
```

- Triple-quoted strings

```
In [3]: """Python is great!
Python is great!"""
```

```
Out[3]: 'Python is great!\nPython is great!'
```

What do we know?

We know we can add, multiply and compare strings:

```
In [4]: 'Python is great' + '!!!'
```

```
Out[4]: 'Python is great!!!'
```

```
In [5]: 'Python' * 3
```

```
Out[5]: 'PythonPythonPython'
```

```
In [6]: 'Python' == 'python'
```

```
Out[6]: False
```

The `in` operator

We can easily check if one string occurs within another string with the `in` operator:

```
In [7]: s = 'Python is great!'
        'great' in s
```

```
Out[7]: True
```

```
In [8]: 'terrible' in s
```

```
Out[8]: False
```

Strings are sequences

Similar to other types of objects we are going to meet in this course, strings are *sequences*. In case of strings, these are sequences of characters.

One important property of sequences is that they are ordered. This property makes two important operations possible:

- Indexing
- Slicing

Indexing

Indexing is the operation of accessing an element (in this case, a character) in a sequence by its index, i.e. its position.

Confusingly, Python (as many other programming languages) **starts indexing at 0**:

```
In [9]: s[0], s[1], s[2], s[14], s[15]
```

```
Out[9]: ('P', 'y', 't', 't', '!!')
```

Using a too large index results in an `IndexError` :

```
In [10]: s[16]
```

```
-----  
IndexError                                Traceback (most recent call last)  
Cell In [10], line 1  
----> 1 s[16]  
  
IndexError: string index out of range
```

Some questions

1. How do we access the last character

```
In [11]: s[len(s) - 1]
```

```
Out[11]: 'l'
```

1. How do we access the last but one element?

```
In [12]: s[len(s) - 2]
```

```
Out[12]: 't'
```

Indexing strings from the end is so common that there is a handy shortcut for this:

```
In [13]: s[-1] == s[len(s) - 1]
```

```
Out[13]: True
```

```
In [14]: s[-2] == s[len(s) - 2]
```

```
Out[14]: True
```

Again, going too far back will throw an `IndexError`.

```
In [15]: s[-17]
```

```
-----  
IndexError                                Traceback (most recent call last)  
Cell In [15], line 1  
----> 1 s[-17]  
  
IndexError: string index out of range
```

Another question

How to prevent the `IndexError` ?

```
In [16]: def index_safe(s, ind):  
         if -len(s) <= ind < len(s):  
             return s[ind]  
         else:  
             return None
```

Slicing

Indexing allows us to access one character at a time. With slicing we can retrieve **ranges** of characters between two indices.

As before, indices start from 0 but to add even more fun, the final index **is not included** in the returned character range:

```
In [17]: s[0:6], s[7:9], s[10:len(s)]
```

```
Out[17]: ('Python', 'is', 'great!')
```

Negative indices work as before:

```
In [18]: s[-9:-7], s[7:-7]
```

```
Out[18]: ('is', 'is')
```

In addition, leaving any of the indices out counts as start and end of string respectively:

```
In [19]: s[:11] == s[0:11]
```

```
Out[19]: True
```

```
In [20]: s[7:] == s[7:len(s)]
```

```
Out[20]: True
```

Unlike with indexing, taking too large a slice does not result in an error:

```
In [21]: s[-1000:1000]
```

```
Out[21]: 'Python is great!'
```

Question

What does `s[:]` return?

Strings are immutable

That means we cannot modify a string after it has been created:

```
In [22]: s[0] = 'p'
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In [22], line 1  
----> 1 s[0] = 'p'  
  
TypeError: 'str' object does not support item assignment
```

```
In [23]: s[:6] = 'Perl'
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In [23], line 1  
----> 1 s[:6] = 'Perl'  
  
TypeError: 'str' object does not support item assignment
```

But what if we want to modify a string?

Well, that's not possible. However, we can make a new string using slicing and concatenation:

```
In [24]: s[:10] + 'definitely not ' + s[-6:]
```

```
Out[24]: 'Python is definitely not great!'
```

However, that still does not modify the old string.

```
In [25]: s
```

```
Out[25]: 'Python is great!'
```

If we really want `s` to have the new value, we need to assign it explicitly.

```
In [26]: s = s[:10] + 'definitely not ' + s[-6:]  
s
```

```
Out[26]: 'Python is definitely not great!'
```

Iteration

We can easily iterate over strings with for loops:

```
In [27]: for x in 'Python is great!':  
         print(x)
```

P
y
t
h
o
n

i
s

g
r
e
a
t
!

For instance, here is a function which counts the number of occurrences of a character in a string

```
In [28]: def count_occur(text, char):  
         n = 0  
         for c in text:  
             if c == char:  
                 n += 1  
         return n  
  
count_occur('Python is great', 't')
```

Out[28]: 2

String methods

Strings have many specialised methods. For instance, you can check whether a string starts with a particular prefix:

```
In [29]: s = 'disfunctional'  
         s.startswith('dis')
```

Out[29]: True

We can also create an upper-case copy of a string:

```
In [30]: s.upper()
```

```
Out[30]: 'DISFUNCTIONAL'
```

Or replace a part of a string with another string:

```
In [31]: s.replace('dis', '')
```

```
Out[31]: 'functional'
```

There are many more (e.g. `strip`), we'll pick them up as we go.

String interpolation

A relatively new, but very convenient, feature in Python is called **formatted string literals**. In other programming languages, this is often referred to as **string interpolation**. This allows you to mix Python code into your string literals. The same thing could be achieved in various other ways, including the `str.format` method, the (now rather obsolete) `%` operator with strings, or manual type conversions and string concatenation with the `+` operator. Let us demonstrate all of these ways, using a simple example of obtaining a name of the form "SURNAME, M. First" given the variables `first_name`, `middle_name` and `surname`. For instance:

```
In [32]: first_name = "Harry"
         middle_name = "Something"
         surname = "Truman"
```

You can "manually" compute the desired result, but this gets a bit cumbersome:

```
In [33]: surname.upper() + ", " + first_name + " " + middle_name[0] + "."
```

```
Out[33]: 'TRUMAN, Harry S.'
```

The formerly recommended way in Python was to use the `str.format` method, where the `{}` in the string are replaced by the corresponding argument to the `format` method:

```
In [34]: "{} {}, {}".format(surname.upper(), first_name, middle_name[0])
```

```
Out[34]: 'TRUMAN, Harry. S'
```

There is in fact yet another way of doing the same thing, using the `%` operator on strings. This is considered obsolete, but you may see it in older Python code:

```
In [35]: "%s, %s %s." % (surname.upper(), first_name, middle_name[0])
```

```
Out[35]: 'TRUMAN, Harry S.'
```

Using formatted string literals, you can make this more readable. Note the `f` right before the first quotation mark:

```
In [36]: f"{surname.upper()}, {first_name} {middle_name[0]}."
```

```
Out[36]: 'TRUMAN, Harry S.'
```

It is also possible to use format specifiers to for instance pad sting with spaces, or round floating-point values. These are given after the Python expression, and begin with a colon `:` sign, for instance:

```
In [37]: f"pi is {355/113:.5f}"
```

```
Out[37]: 'pi is 3.14159'
```

The `.5f` part means that the value should be printed as a decimal number with five decimals. There is a very powerful and complex system for specifying how to format strings, which is fully described in the Python reference manual:

- [Python format specification language](#)

Lists and tuples

Similar to strings, lists and tuples are **ordered sequences of elements**.

However, while the elements in strings could only be characters, lists and tuples allow mixing different types of elements:

```
In [38]: ['I', 'love', 'Python', 3]
         (1, 2, 3, 'four', 5)
         [[0], 1, ('pi', 3.14)]
```

```
Out[38]: [[0], 1, ('pi', 3.14)]
```

We indicate lists with square brackets and tuples with parentheses.

Old stuff works

Addition and multiplication work as expected, given what we know about strings:

```
In [39]: [1, 2, 3] + [4, 5]
```

```
Out[39]: [1, 2, 3, 4, 5]
```

```
In [40]: (1, 2, 3) + (4, 5)
```

```
Out[40]: (1, 2, 3, 4, 5)
```

```
In [41]: [1, 2] * 3
```

```
Out[41]: [1, 2, 1, 2, 1, 2]
```

```
In [42]: (1, 2) * 3
```

```
Out[42]: (1, 2, 1, 2, 1, 2)
```

We cannot mix types, though:

```
In [43]: [1, 2, 3] + (4, 5)
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In [43], line 1  
----> 1 [1, 2, 3] + (4, 5)  
  
TypeError: can only concatenate list (not "tuple") to list
```

```
In [44]: [1, 2, 3] + 4
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In [44], line 1  
----> 1 [1, 2, 3] + 4  
  
TypeError: can only concatenate list (not "int") to list
```

To get the number of elements we can use `len()` :

```
In [45]: len([1, 2, 3])
```

```
Out[45]: 3
```

```
In [46]: len((1, 2, 3))
```

```
Out[46]: 3
```

What is the length of the following list?

```
In [47]: l = [('I', 'PRON'), ('love', 'VERB'), ('Python', 'NOUN')]  
len(l)
```

```
Out[47]: 3
```

Indexing and slicing

Not much new here:

```
In [48]: l = ['I', 'love', 'Python']  
l[0]
```

```
Out[48]: 'I'
```

```
In [49]: l[-1]
```

```
Out[49]: 'Python'
```

```
In [50]: l[3]
```

```
-----  
IndexError                                Traceback (most recent call last)  
Cell In [50], line 1  
----> 1 l[3]  
  
IndexError: list index out of range
```

```
In [51]: l[:2]
```

```
Out[51]: ['I', 'love']
```

```
In [52]: l[-1:]
```

```
Out[52]: ['Python']
```

```
In [53]: l[-1]
```

```
Out[53]: 'Python'
```

Indexing & slicing embedded lists

Since lists and tuples can contain other lists and tuples (which can in turn contain yet other lists and tuples), we need to chain several indices to get to embedded lists:

```
In [54]: l = [('I', 'PRON'), ('love', 'VERB'), ('Python', 'NOUN')]  
         l[0]
```

```
Out[54]: ('I', 'PRON')
```

```
In [55]: l[:2]
```

```
Out[55]: [('I', 'PRON'), ('love', 'VERB')]
```

```
In [56]: l[0][0]
```

```
Out[56]: 'I'
```

```
In [57]: l[0][1][-2:]
```

Out[57]: 'ON'

How do we get the last two phonetic segments in 'Python' ?

```
In [58]: l = [('I', 'PRON', 'aI'),  
             ('love', 'VERB', 'lVv'),  
             ('Python', 'NOUN', 'paIT@n')]
```

```
In [59]: l[2][2][-2:]
```

Out[59]: '@n'

Iteration

We can iterate over lists and tuples as easily as over strings:

```
In [60]: l = ['I', 'love', 'Python']  
  
for elem in l:  
    print(elem)
```

I
love
Python

What does this do?

```
In [61]: def some_function(wordlist):  
  
    result = []  
  
    for w in wordlist:  
        if w.endswith('ly'):  
            result.append(w[:-2])  
        else:  
            result.append(w)  
  
    return result
```

```
In [62]: l = ['fast', 'neatly', 'boldly']  
# Try to guess before you run!  
# some_function(l)
```

Lists are mutable

Unlike strings (and tuples), lists are mutable. That means, we can modify an existing list:

```
In [63]: l = [1, 2, 3, (4, 5), 6]  
l[0] = -99  
l
```

```
Out[63]: [-99, 2, 3, (4, 5), 6]
```

```
In [64]: l[1:3] = [0, 0]  
l
```

```
Out[64]: [-99, 0, 0, (4, 5), 6]
```

```
In [65]: l[-2] = 'seven'  
l
```

```
Out[65]: [-99, 0, 0, 'seven', 6]
```

```
In [66]: l.append(7)  
l
```

```
Out[66]: [-99, 0, 0, 'seven', 6, 7]
```

Destructive functions

What is the result of the following operations?

```
In [67]: l = [1, 2]  
l = l.append(3)  
print(l)
```

None

Most functions we have seen so far returned **copies** of the objects passed as arguments.

`append()` behaves differently, it changes the list **in place** and **returns no value**. That is why assigning the result to the original variable results in `None`. This is a common error, which confuses two possible ways of achieving the same thing:

```
In [68]: l = [1,2]
         l = l + [3]
         print(l)
```

[1, 2, 3]

```
In [69]: l = [1,2]
         l.append(3)
         print(l)
```

[1, 2, 3]

Equality and identity

We can ask Python to create two list objects and bind each to a variable:

```
In [70]: l1 = ['I', 'love', 'Python']
         l2 = ['I', 'love', 'Python']
```

These two lists are equal since they have the same elements:

```
In [71]: l1 == l2
```

Out[71]: True

However, they are not identical since they refer to two different objects:

```
In [72]: l1 is l2
```

Out[72]: False

If we modify `l1`, `l2` will not change:

```
In [73]: l1[1] = 'hate'
l1, l2
```

```
Out[73]: (['I', 'hate', 'Python'], ['I', 'love', 'Python'])
```

We can also make two variables refer to the same object.

```
In [74]: l1 = ['I', 'love', 'Python']
l2 = l1
l1 is l2
```

```
Out[74]: True
```

Now, because `l1` and `l2` refer to the same object, modifying the value of one will also modify the value of the other:

```
In [75]: l1[1] = 'hate'
l1, l2
```

```
Out[75]: (['I', 'hate', 'Python'], ['I', 'hate', 'Python'])
```

If the above is not crystal clear to you, we strongly recommend you to run the examples above in the [Python Visualizer](#) to see how the different objects are stored separately in memory and modified individually.

Copies and references

This behaviour is quite unexpected at first and can cause **a lot of harm** if we are not careful.

This might happen, for instance, when we modify a list argument inside the function:

```
In [76]: l = [1,2,3]
x = l.pop()
```

```
In [77]: def last_elem(x):
return x.pop()
```

```
l = [1, 2, 3]
last_elem(l)
```

Out[77]: 3

At this point, we might be surprised to see that the value of `l` has changed as well:

```
In [78]: l
```

Out[78]: [1, 2]

For this reason, it is often a good idea to create a copy of the original object using the empty slicing operator:

```
In [79]: l1 = ['I', 'love', 'Python']
        l2 = l1[:] # l2 is now a copy of l1
        l1 is l2
```

Out[79]: False

```
In [80]: l1[1] = 'hate'
        l1, l2
```

Out[80]: (['I', 'hate', 'Python'], ['I', 'love', 'Python'])

Note that since lists may contain other lists, this does not protect you from having *those* lists modified. For instance:

```
In [81]: l1 = [['First', 'sentence'], ['Second', 'sentence']]
        l2 = l1[:]
        l2[0][1] = 'orangutan'
        l1, l2
```

Out[81]: ([['First', 'orangutan'], ['Second', 'sentence']],
 [['First', 'orangutan'], ['Second', 'sentence']])

There are two ways to avoid this type of problem:

1. Prefer to use immutable data types, like tuples.
2. If you need to use mutable data types, like lists, use the `[`copy.deepcopy` function]` (<https://docs.python.org/3/library/copy.html>).

Unfortunately, both of the above may lead to unnecessary duplication of data which wastes memory and time. Using shared mutable data structures is often more efficient. This is an example of the general tendency in programming where efficiency is often at odds with safety.

Splitting and joining strings

Using the `split()` method we can turn a string into a list by breaking it up on an arbitrary delimiter character:

```
In [82]: s = 'I love Python!'
s.split(' ')
```

```
Out[82]: ['I', 'love', 'Python!']
```

We can also join back a list of strings (and only strings) using the `join` method:

```
In [83]: l = ['I', 'love', 'Python']
' '.join(l)
```

```
Out[83]: 'I love Python'
```

```
In [84]: print('\n'.join(l))
```

```
I
love
Python
```

Note that `join` is a string method not a list method!

```
In [85]: ','.join(['1', '2', '3'])
```

```
Out[85]: '1,2,3'
```

Tuples

Tuples are very similar to lists but, like strings, they are immutable:

```
In [86]: t = (1, 2, 3)
         t[0] = -99
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In [86], line 2
      1 t = (1, 2, 3)
----> 2 t[0] = -99

TypeError: 'tuple' object does not support item assignment
```

Tuples are useful when we want to ensure that an object is not going to change during the execution of the program, as in this example:

```
In [87]: l = [('I', 'PRON', 'aI'),
              ('love', 'VERB', 'lVv'),
              ('Python', 'NOUN', 'paIT@n')]
```

Comprehensions

In Python, there is a powerful mechanism to create a new collection data type (including lists and tuples) as a loop. To give a motivating example, you might want to create a list of backwards words given a list of the original words:

```
In [88]: def reverse_words(words):
         backwards = []
         for word in words:
             backwards.append(word[::-1])
         return backwards

reverse_words("Hello there everyone".split())
```

```
Out[88]: ['olleH', 'ereht', 'enoyreve']
```

Using a **list comprehension** we can express the same thing in a much more compact way and without defining the temporary variable `backwards` :

```
In [89]: def reverse_words(words):
         return [word[::-1] for word in words]

reverse_words("Hello there everyone".split())
```

```
Out[89]: ['olleH', 'ereht', 'enoyreve']
```

List comprehensions also allow filtering, specified by an `if` at the end of the expression:

```
In [90]: def reverse_words(words):  
         return [word[::-1] for word in words if word[0].islower()]  
  
reverse_words("Hello there everyone".split())
```

```
Out[90]: ['ereht', 'enoyreve']
```

Comprehensions also work with tuples:

```
In [91]: def reverse_words(words):  
         return tuple(word[::-1] for word in words if word[0].islower())  
  
reverse_words("Hello there everyone".split())
```

```
Out[91]: ('ereht', 'enoyreve')
```

Note that here the input data type is still a `list` (what we get from the `.split` method, just like in the earlier examples), but this does not prevent us from creating a tuple.

Comprehensions with strings are not supported directly, but you can easily simulate this using list comprehensions and the `join` method:

```
In [92]: def caesar_decrypt(ciphertext):  
         return ''.join([chr(ord(letter)-1) for letter in ciphertext])  
  
caesar_decrypt("IBM:111")
```

```
Out[92]: 'HAL9000'
```

The built-in `ord` and `chr` functions are used to convert between characters (strings of length 1) and integers, and back. The function creates a new string just like the input string, but always picks one letter *before*. This is similar to the famous Caesar cipher.

Now, let us look at almost the same code again. Can you spot the difference?

```
In [93]: def caesar_decrypt(ciphertext):  
         return ''.join(chr(ord(letter)-1) for letter in ciphertext)  
  
         caesar_decrypt("IBM:111")
```

```
Out[93]: 'HAL9000'
```

There is no list comprehension here. Instead, we use a syntactically very similar **generator expression**. The details are beyond the scope of this course, but the advantage here is that we do not need to create an intermediate list containing all the letters in the output. Instead, the `join` method is fed the characters one by one from the generator expression. In this case it does not matter, but in the lecture on files we will see how this detail can make the difference between a successful program and one that eats all available memory and crashes your computer.

Further examples

Below are some examples using concepts we have learned. Note that these are not necessarily the *best* ways of performing the operations below, but they are reasonable attempts *given what we know so far*.

Example 1

```
In [94]: def strip_punctuation(text):  
         for punct in ".,:":  
             text = text.replace(punct, '')  
         return text
```

Example 2

How long will this take for a text of a thousand words? A million words? We will discuss this in more detail during the next lecture, but it is good to start thinking about it now.

```
In [95]: def text_vocabulary(text):  
         words = strip_punctuation(text).split()  
         words_unique = []  
  
         for w in words:  
             if w.lower() not in words_unique:
```

```
words_unique.append(w.lower())

words_unique.sort()
return words_unique
```

Example 3

How would you rewrite this using a list comprehension?

```
In [96]: def filter_long_lines(text, min_len=4):
lines = text.split('\n')
long = []

for l in lines:
    words = l.split()

    if len(words) >= min_len:
        long.append(l)

return '\n'.join(long)
```